
Melchior

Project Documentation: Algorithmic Trading Backtester

Version 1.0

Author: Antonio Machuca

Date: June 12, 2026

Contents

1	Introduction	2
1.1	Motivation	2
1.2	Purpose and Objectives	2
1.3	Scope	2
1.4	Acronyms and Abbreviations	2
1.5	References	2
2	Theoretical Foundations	4
2.1	Financial Time Series Modeling	4
2.2	Trading Strategies	7
2.2.1	Simple Moving Average (SMA)	7
2.2.2	Momentum	7
2.3	Performance Metrics	7
3	Architecture and Structural Design	9
3.1	System Architecture	9
3.2	UML Class Design	9
3.3	Diagrams	9
4	Implementation and Vectorization	10
4.1	Vectorization over Iteration	10
4.2	Structured Programming	10
4.3	Friction Models	10
5	Results and Conclusions	11
5.1	Backtest Execution Analysis	11
5.2	Conclusions	11
5.3	Future Work	11

1 Introduction

1.1 Motivation

Algorithmic trading requires processing vast amounts of historical market data. This project is motivated by the need for a robust, immutable, and highly efficient infrastructure that completely eliminates these procedural risks through strict mathematical vectorization.

1.2 Purpose and Objectives

The primary objective of this document is to detail the design and implementation of a high-performance backtesting engine. The central goal is to achieve an effective asymptotic time complexity of $\mathcal{O}(1)$ for financial array operations. By strictly adhering to structured programming paradigms, avoiding loops and control flow interruptions, the engine ensures deterministic execution and scalable quantitative analysis.

1.3 Scope

The scope of this software focuses exclusively on the execution and evaluation of trading algorithms. It enforces a clear separation of concerns: the core algorithmic and mathematical logic is securely encapsulated within the domain (`src/core/`), while all interactive data exploration, charting, and visual analyses are isolated in the presentation layer (`notebooks/`).

1.4 Acronyms and Abbreviations

- **API:** Application Programming Interface.
- **SMA:** Simple Moving Average.
- **GBM:** Geometric Brownian Motion.
- **MDD:** Maximum Drawdown.
- **CAGR:** Compound Annual Growth Rate.

1.5 References

- **Title:** Python for Finance: Mastering Data-Driven Finance (2nd Edition).

- **Author:** Yves Hilpisch.
- **Publishing Organization:** O'Reilly Media.
- **Date:** 2018.
- **ISBN/DOI:** 978-1492024330.
- **Title:** Advances in Financial Machine Learning.
 - **Author:** Marcos López de Prado.
 - **Publishing Organization:** John Wiley & Sons.
 - **Date:** 2018.
 - **ISBN/DOI:** 978-1119482086.
- **Title:** Options, Futures, and Other Derivatives (10th Edition).
 - **Author:** John C. Hull.
 - **Publishing Organization:** Pearson.
 - **Date:** 2017.
 - **ISBN/DOI:** 978-0134472089.

2 Theoretical Foundations

This chapter establishes the strict mathematical and stochastic foundations upon which the backtesting engine is built. We present the concepts as formal definitions and observations to ensure clarity.

2.1 Financial Time Series Modeling

Definition 2.1 (Scale Invariance). In financial mathematics, a metric is *scale-invariant* if its significance does not depend on the absolute magnitude of the underlying variable.

Observation 2.1. Therefore, the absolute difference in an asset's price ($P_t - P_{t-1}$) is not scale-invariant.

Example 2.1 (Lack of Scale Invariance). A \$1 drop in a \$10 stock represents a catastrophic 10% loss of capital. However, exactly the same absolute \$1 drop in a \$1000 stock is merely 0.1% market noise. This illustrates why absolute price differences are useless for universal comparisons.

Observation 2.2 (Need for Relative Returns). Because raw price changes lack scale invariance, quantitative models must standardize measurements to evaluate risk and return universally, regardless of the underlying asset price. This forces us to use relative returns rather than absolute price differences.

Definition 2.2 (Asset Price and Returns). Let P_t be the price of a financial asset at a discrete time step t . Based on the necessity for scale invariance, we define two main types of relative returns:

- *Arithmetic (Simple) Return:* $R_t = \frac{P_t - P_{t-1}}{P_{t-1}}$
- *Logarithmic (Continuously Compounded) Return:* $r_t = \ln\left(\frac{P_t}{P_{t-1}}\right)$

Definition 2.3 (Continuous-Time Stochastic Process). A continuous-time stochastic process is a collection of random variables $\{X_t\}_{t \in T}$ where the index set T is a continuous interval (e.g., $T = [0, \infty)$). Each random variable X_t represents the state of the process at time t .

Definition 2.4 (Wiener Process / Brownian Motion). A Wiener process W_t is a continuous-time stochastic process characterized by three main properties:

1. $W_0 = 0$
2. It has independent increments: for every $t > 0$, the future increment $W_{t+u} - W_t$ is independent of the past values W_s ($s \leq t$).

3. It has normally distributed increments: $W_{t+u} - W_t \sim \mathcal{N}(0, u)$.

To ground the concept of a Wiener process (also called Brownian motion), we will give the canonical example, the pollen grain:

Example 2.2. Let there be a microscopic pollen grain suspended in stagnant water. The grain starts at a known position ($W_0 = 0$). Invisible water molecules constantly hit it from random directions, as the collisions are random, every microscopic movement of the grain is independent of its previous movements (independent increments). As time passes, the area where the grain could have gone becomes wider, and the possible locations form a Gaussian bell curve (normally distributed increments).

Definition 2.5 (Itô Drift-Diffusion Process). An Itô drift-diffusion process is a continuous-time stochastic process X_t that combines a deterministic drift component and a stochastic diffusion component driven by a Wiener process. It is mathematically expressed by the Stochastic Differential Equation:

$$dX_t = \mu_t dt + \sigma_t dW_t$$

Where μ_t represents the expected instantaneous drift rate, σ_t represents the instantaneous volatility (or diffusion), and W_t is a standard Wiener process.

To intuitively understand the concept of an Itô drift-diffusion process we will look at an example, the example of the restless dog.

Example 2.3. Consider a person walking a restless dog. The steady and predictable step of the person forward represents the *drift* ($\mu_t dt$). Meanwhile, the dog pulls the leash randomly from side to side to sniff things, which represents the *diffusion* ($\sigma_t dW_t$). The actual path they walk together (dX_t) is the sum of the deterministic direction of the person and the random movements of the dog.

Proposition 2.1 (Itô's Lemma). *If X_t is an Itô drift-diffusion process given by $dX_t = \mu_t dt + \sigma_t dW_t$, and $f(t, x)$ is a twice-differentiable scalar function, then $f(t, X_t)$ is also an Itô process, and its differential is given by:*

$$df = \left(\frac{\partial f}{\partial t} + \mu_t \frac{\partial f}{\partial x} + \frac{1}{2} \sigma_t^2 \frac{\partial^2 f}{\partial x^2} \right) dt + \sigma_t \frac{\partial f}{\partial x} dW_t$$

Observation 2.3. Itô's Lemma is the stochastic equivalent of the chain rule in classical calculus.

Definition 2.6 (Geometric Brownian Motion). The Geometric Brownian Motion or GBM is a continuous-time stochastic process frequently used to model the price paths of financial

assets. It is defined by the Stochastic Differential Equation (SDE):

$$dS_t = \mu S_t dt + \sigma S_t dW_t$$

Where S_t is the asset price at time t , μ is the percentage drift (expected return), σ is the percentage volatility, and W_t is a standard Wiener process (Brownian motion). Applying Itô's Lemma, it can be shown that under this model, the natural logarithm of the asset price follows a normal distribution.

Observation 2.4 (Arithmetic vs. Logarithmic Returns). Logarithmic returns are widely used in quantitative analysis because they are time-additive: the total return over n days is the sum of the daily logarithmic returns ($r_{0,n} = \sum_{t=1}^n r_t$). Furthermore, under the GBM model, logarithmic returns are normally distributed, which facilitates statistical modeling.

However, *logarithmic returns are not cross-sectionally additive* (the sum of the logarithmic returns of two assets does not equal the logarithmic return of a portfolio containing them). Therefore, when simulating the actual **equity curve of a portfolio**, we must strictly use arithmetic returns (R_t) combined with cumulative products.

$$V_T = V_0 \prod_{t=1}^T (1 + R_{\text{strategy},t} - C_t)$$

Where V_T is the portfolio value at time T , and C_t represents the transaction costs.

Proposition 2.2 (Temporal Scaling of Volatility - Annualization). *Assuming that the logarithmic returns r_t are independent and identically distributed (i.i.d.) random variables, the variance of the returns scales linearly with time.*

$$\text{Var}(r_{\text{annual}}) = 252 \cdot \text{Var}(r_{\text{daily}})$$

Where 252 is the standard number of trading days in a year.

Observation 2.5. Since volatility (σ) is defined as the standard deviation (the square root of the variance), taking the square root of both sides of the previous proposition yields:

$$\sigma_{\text{annual}} = \sqrt{\text{Var}(r_{\text{annual}})} = \sqrt{252 \cdot \text{Var}(r_{\text{daily}})} = \sqrt{252} \cdot \sigma_{\text{daily}}$$

This explains why we scale risk (volatility) using the square root of time, whereas expected returns scale linearly by simply multiplying the daily mean by 252.

2.2 Trading Strategies

2.2.1 Simple Moving Average (SMA)

Definition 2.7 (Simple Moving Average). The simple moving average or SMA over a lookback window of n periods at time t is the unweighted mean of the previous n data points:

$$SMA_t^{(n)} = \frac{1}{n} \sum_{i=0}^{n-1} P_{t-i}$$

Observation 2.6. A classic strategy generates a bullish signal (+1) when a short-term SMA crosses above a long-term SMA, and a bearish signal (−1) otherwise.

2.2.2 Momentum

Definition 2.8 (Momentum). Momentum measures the rate of change in an asset's price over a specific period k . It is mathematically defined as the arithmetic return over k periods:

$$M_t^{(k)} = \frac{P_t - P_{t-k}}{P_{t-k}}$$

Observation 2.7. The strategy operates as follows: it buys when the momentum is strictly positive, i.e. $M_t^{(k)} > 0$, and sells when it is negative.

2.3 Performance Metrics

Definition 2.9 (Sharpe Ratio). The Sharpe Ratio measures the excess return per unit of total risk (volatility).

$$S = \frac{\mathbb{E}[R_p - R_f]}{\sigma_p}$$

Where R_p is the portfolio return and R_f is the risk-free rate.

Observation 2.8 (Limitations of the Sharpe Ratio). A major mathematical flaw of the Sharpe Ratio is that the standard deviation (σ_p) treats both upward volatility (massive sudden gains) and downward volatility (losses) equally as "risk". A rational investor does not fear positive volatility. To correct this, the *Sortino Ratio* is often preferred, as it only measures the semi-variance of negative returns (downside risk).

Definition 2.10 (Maximum Drawdown and High-Water Mark). The Maximum Drawdown (MDD) is the largest drop from a historical peak in the portfolio's value. To define it, we first define the *High-Water Mark* (the running maximum equity up to time t):

$\max_{\tau \leq t} V_\tau$. The MDD is the worst relative drop:

$$MDD = \min_t \left(\frac{V_t - \max_{\tau \leq t} V_\tau}{\max_{\tau \leq t} V_\tau} \right)$$

Observation 2.9 (Asymmetry of Drawdowns). The MDD is critical because financial recovery is strictly asymmetric. If a portfolio suffers a 50% drop, it requires a subsequent 100% gain just to reach the break-even point. This is why accurately tracking the running maximum reliably reflects the true "risk of ruin".

3 Architecture and Structural Design

3.1 System Architecture

Description of the Hexagonal Architecture pattern (Ports and Adapters) and SOLID principles (ISP and DIP) applied to decouple the data ingestion from the core logic.

3.2 UML Class Design

Detailed structural explanation of the core domain classes:

- `DataHandler` (Interface and Adapters)
- `Strategy` (Strategy Pattern)
- `Portfolio` (Aggregate Root)
- `BacktestEngine` (Orchestrator)

3.3 Diagrams

(Space to include the UML Class Diagram `ClassDiagram.svg` generated in the `docs/architecture/` directory).

4 Implementation and Vectorization

4.1 Vectorization over Iteration

Explain the strict paradigm of avoiding `for` or `while` loops. Detail how `pandas` and `NumPy` were used to achieve $\mathcal{O}(1)$ asymptotic complexity for array operations across time-series data.

4.2 Structured Programming

Detail the adherence to academic structured programming paradigms, including the absolute prohibition of control flow interruption statements like `break` or `continue`.

4.3 Friction Models

Describe the implementation of the `IFrictionModel` for transaction costs calculation without loops, strictly using dataframe shift operations.

5 Results and Conclusions

5.1 Backtest Execution Analysis

Analysis of the performance and execution time. Showcase the output of the Jupyter notebook visualizations (`matplotlib` and `plotly`) plotting the *Equity Curve*.

5.2 Conclusions

Summary of the project's achievements against its initial goals in terms of software architecture, algorithmic efficiency, and mathematical rigor.

5.3 Future Work

Potential improvements, such as adding new statistical models, expanding the framework to multi-asset portfolios, or integrating real-time live trading connectivity.